

Product & Technical Specification

Domain model, authorization, API boundaries, modular architecture and event-driven direction

1. Domain Model

Initial hierarchy: User -> Organization -> Project -> Task -> Comment. Organization membership and permissions define the primary tenant boundary. Project membership provides project-level authorization.

2. Conceptual Relationships

Entity	Relationship	Notes
User	belongs to organizations	Can participate in organizations under the initial organization-based model.
Organization	contains members and projects	Primary tenant boundary.
Membership	connects user and organization	Carries organization-level role.
Project	belongs to organization	Project-level permission boundary.
Project membership	connects user and project	Supports project-level role/permissions.
Task	belongs to project	Core unit of work.
Comment	belongs to task	Collaboration record.
File metadata	future association	Metadata only initially; actual storage is future scope.

3. Authorization Model

Every protected operation should evaluate: User -> Organization membership -> Project membership/role -> Permission -> Resource -> Action. A resource ID alone never establishes access.

4. Recommended Modular Monolith

Use a modular monolith rather than microservices initially. This keeps the project understandable while still allowing meaningful use of Redis, Kafka, background workers and observability.

5. Logical Layers

- HTTP/API layer: routing, request parsing, validation, authentication middleware and response formatting.
- Application/service layer: use cases, orchestration, authorization and transaction boundaries.
- Domain layer: entities, business rules and domain events where appropriate.
- Repository/data layer: PostgreSQL persistence.
- Infrastructure adapters: Redis, Kafka, email, metrics and tracing.
- Workers: asynchronous consumers and maintenance jobs.

6. Event-Driven Direction

Topic	Example events	Consumers
user.events	registered, verified, password-reset-requested	Notification worker; future audit/analytics
organization.events	member-invited, member-added	Notification worker
project.events	created, updated, archived	Future analytics/consumers

Topic	Example events	Consumers
task.events	created, assigned, status-changed, due-soon	Notification and maintenance workers
comment.events	created, updated, deleted, mentioned	Notification worker
notification.events	email-requested, delivery-failed	Email delivery/retry consumers

7. Redis Direction

- Use namespaced keys and explicit TTLs.
- Keep rate-limit keys separate from session/token keys.
- Cache only data with a clear invalidation strategy.
- Distributed locks must have bounded TTLs and failure-safe behavior.
- Define Redis failure behavior separately for each use case.

8. Tenant Isolation

- Every organization-scoped resource must be traceable to its organization.
- Service methods establish organization context before protected reads/writes.
- Repository queries should include tenant constraints where appropriate as defense in depth.
- Integration tests must attempt cross-tenant access and assert rejection.

9. Initial Endpoint Inventory

Area	Initial direction
Auth	/api/v1/auth/register, /login, /refresh, /logout, /verify-email, /forgot-password, /reset-password
Organizations	/api/v1/organizations, /api/v1/organizations/{id}, /members, /invitations
Projects	/api/v1/projects, /api/v1/projects/{id}, /members
Tasks	/api/v1/projects/{projectId}/tasks, /api/v1/tasks/{id}
Comments	/api/v1/tasks/{taskId}/comments, /api/v1/comments/{id}
Query	Collection endpoints support page/limit, filters, sort and search.
Health	/health, /ready
Metrics	/metrics

10. Future Evolution

If commercialization occurs, add object storage, 2FA, OAuth/SSO, billing, usage limits, audit logs, richer membership models, advanced analytics and specialized search infrastructure only when justified.